



Programmation fonctionnelle (OCAML)

Mouhamadou GAYE

Enseignant-Chercheur
Département d'Informatique
Master en Informatique
Option Génie Logiciel

19 mai 2025



Chapitre 2 : Structures de données



Définition

Une liste est une structure délimitée par des crochets contenant des éléments de même type séparés les uns des autres par un point-virgule.

Exemple :

```
let l = [4 ; 3 ; 2 ; 1] ;;  
val l : int list = [4 ; 3 ; 2 ; 1]
```



- **Construction**

- Une liste vide est de type 'a list et représentée par [] ;



- **Construction**

- Une liste vide est de type 'a list et représentée par [] ;
- L'opérateur infixe :: appelé conse ajoute un élément en tête de liste ;



- **Construction**

- Une liste vide est de type 'a list et représentée par [] ;
- L'opérateur infixe :: appelé conse ajoute un élément en tête de liste ;
- L'opérateur infixe @ permet de concaténer deux listes.



- Quelques fonctions du module **List**
 - **List.hd** et **List.tl** donnent respectivement le premier élément et la liste privée de son premier élément ;



- Quelques fonctions du module **List**

- **List.hd** et **List.tl** donnent respectivement le premier élément et la liste privée de son premier élément ;
- **List.mem** x **L** : qui teste l'appartenance d'un élément x à une liste **L** ;



- Quelques fonctions du module **List**

- **List.hd** et **List.tl** donnent respectivement le premier élément et la liste privée de son premier élément ;
- **List.mem x L** : qui teste l'appartenance d'un élément x à une liste L ;
- **List.nth n L** : qui renvoie le n ème élément d'une liste ou lève l'exception `Liste_vide` si il n'existe pas (le compteur commence à 0) ;



- Quelques fonctions du module **List**

- **List.hd** et **List.tl** donnent respectivement le premier élément et la liste privée de son premier élément ;
- **List.mem x L** : qui teste l'appartenance d'un élément x à une liste L ;
- **List.nth n L** : qui renvoie le n ème élément d'une liste ou lève l'exception `Liste_vide` si il n'existe pas (le compteur commence à 0) ;
- **List.append L1 L2** : concatène les listes $L1$ et $L2$;



- Quelques fonctions du module **List**

- **List.hd** et **List.tl** donnent respectivement le premier élément et la liste privée de son premier élément ;
- **List.mem** **x** **L** : qui teste l'appartenance d'un élément **x** à une liste **L** ;
- **List.nth** **n** **L** : qui renvoie le nième élément d'une liste ou lève l'exception **Liste_vide** si il n'existe pas (le compteur commence à 0) ;
- **List.append** **L1** **L2** : concatène les listes **L1** et **L2** ;
- **List.rev** **L** : inverse l'ordre des éléments d'une liste **L**.



- Quelques fonctions du module **List**
 - **List.rev_append L1 L2** : inverse l'ordre des éléments de L1 et les ajoute en tête de L2 ;



- Quelques fonctions du module **List**

- **List.rev_append L1 L2** : inverse l'ordre des éléments de L1 et les ajoute en tête de L2 ;
- **List.for_all p L** : vérifie si pour tout élément x de L, $p(x)$ est vraie ;



- Quelques fonctions du module **List**

- **List.rev_append L1 L2** : inverse l'ordre des éléments de L1 et les ajoute en tête de L2 ;
- **List.for_all p L** : vérifie si pour tout élément x de L, $p(x)$ est vraie ;
- **List.iter f L** : applique la fonction f à tous les éléments de la liste L ;



- Quelques fonctions du module **List**

- **List.rev_append L1 L2** : inverse l'ordre des éléments de L1 et les ajoute en tête de L2 ;
- **List.for_all p L** : vérifie si pour tout élément x de L, $p(x)$ est vraie ;
- **List.iter f L** : applique la fonction f à tous les éléments de la liste L ;
- **List.filter p L** : extrait la sous-liste des éléments de L satisfaisant le prédicat p .



Définition

Un tableau est une structure délimitée par `[` et `]` contenant des éléments de même type séparés les uns des autres par un point-virgule.

Exemple :

```
let tab = [| 3; 2; 1 |] ;;  
val tab : int array = [|3; 2; 1|]
```



- La fonction **Array.make** permet de créer un tableau en donnant la taille et la valeur d'initialisation.



- La fonction **Array.make** permet de créer un tableau en donnant la taille et la valeur d'initialisation.
- La fonction **Array.length** donne la taille d'un tableau.



Tableaux

- La fonction **Array.make** permet de créer un tableau en donnant la taille et la valeur d'initialisation.
- La fonction **Array.length** donne la taille d'un tableau.
- **Array.to_list** transforme un tableau en liste.



- La fonction **Array.make** permet de créer un tableau en donnant la taille et la valeur d'initialisation.
- La fonction **Array.length** donne la taille d'un tableau.
- **Array.to_list** transforme un tableau en liste.
- Pour un tableau **t**, **t.(i)** désigne l'élément d'indice **i** du tableau **t**. Il peut être consulté ou modifié avec l'opérateur **<-**.

Exemple :

```
let t = Array.make 2 3;;  
val t : int array = [|3; 3|]  
t.(0) <- 4;;  
- : unit = ()  
t;;  
- : int array = [|4; 3|]
```



Définition

Une énumération est un type défini par la liste de ses différentes valeurs. Chaque valeur possible d'un type énuméré est appelé constructeur qui doit obligatoirement commencer par une majuscule.

Exemple :

```
type couleur =  
  Bleu  
  | Blanc  
  | Rouge;;  
let p = Blanc;;  
p : couleur = Blanc
```



Définition

Le type somme est une énumération dont les constructeurs peuvent être paramétrés par un type pouvant être différent pour chaque constructeur.

Exemple :

```
type nombres =  
  Entier of int  
  | Reel of float;;  
let print_entier = function  
  Entier n -> print_int n  
  | Reel x -> print_float x;;  
print_entier : nombres -> unit = <fun>
```



Définition

Le type produit ou enregistrement est utilisé pour modéliser les données qui ont simultanément plusieurs propriétés. Chacune des caractéristiques est accompagnée de son type respectif et la liste de toutes les caractéristiques est entourée d'accolades `{}`.

Exemple :

```
type monome = { coefficient : int ; degre : int } ; ;  
let m = {coefficient = 1 ; degre = 0} ; ;  
m : monome = {coefficient = 1 ; degre = 0}
```

NB : Les champs d'un enregistrement ne sont pas modifiables par défaut. Pour modifier un champ il faut le déclarer explicitement à l'aide du mot clé `mutable`.

